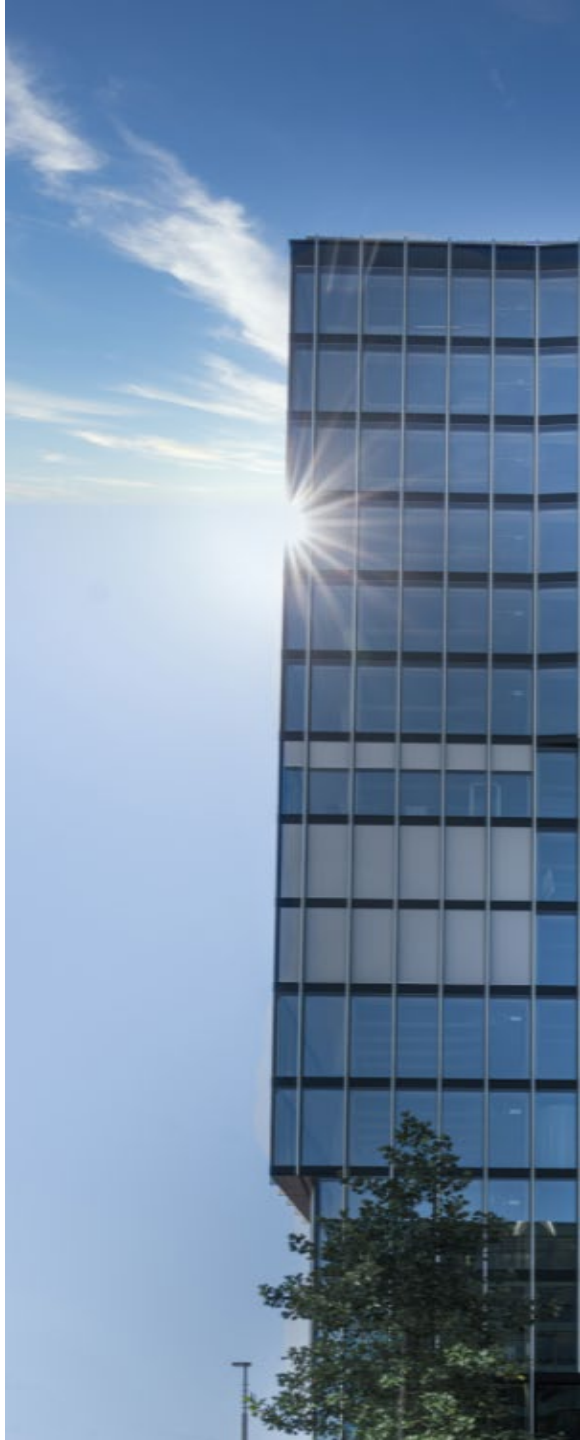
HSLU Hochschule Luzern

Objektorientierte Programmierung

Datenstrukturen: Collections - Grundlagen

Roland Gisler



FH Zentralschweiz

Inhalt

- Was sind Datenstrukturen?
- Eigenschaften von Datenstrukturen
- Java Collection Framework
- Konkretes Beispiel: `ArrayList`
- Auswahl der geeigneten Datenstruktur
- Zusammenfassung

Lernziele

- Sie wissen, was Datenstrukturen sind und wofür man sie einsetzt.
- Sie kennen verschiedene Eigenschaften von Datenstrukturen.
- Sie kennen den groben Umfang des Java Collections Frameworks.
- Sie kennen einige grundlegende Schnittstellen, Algorithmen und Implementationen des Java Collections Frameworks.
- Sie verstehen die Klasse **`ArrayList`** als konkrete Implementation einer **`List`** und können diese nutzen.
- Sie sind in der Lage, eine den Anforderungen angemessene Datenstruktur auszuwählen.

Was sind Datenstrukturen?

Was sind Datenstrukturen?

- Sehr häufig haben wir es in der Programmierung mit Mengen von Daten bzw. Objekten zu tun.
  Beispiele:
  - Messwerte (z.B. `Temperatur`en)
  - Bankkonten (mit Besitzer und Kontostand)
  - Formen (wie z.B. `Circle` oder `Rectangle`)
  - Personen (z.B. Studierende)
- Datenstrukturen werden verwendet, um **Mengen** von Daten bzw. Objekten **effizient zu speichern** und **verarbeiten** zu können.
- Unter Verarbeiten versteht man diverse Funktionen, z.B.
  - alle enthaltenen Daten/Objekte einzeln zu bearbeiten.
  - nach bestimmten Objekten zu suchen oder filtern.
  - nach unterschiedlichen Kriterien zu sortieren.

Verschiedene Arten von Datenstrukturen

- Es gibt verschiedene Arten von Datenstrukturen. Beispiele:
  - **Array** – Indexierte Folge/Reihung von Elementen.
  - **List** – Einfache Folge (ohne Index).
  - **Map** – Zuordnung zwischen Schlüssel/Wert-Paaren.
  - **Tree** – Hierarchisch geordnete Daten (Baumstruktur).
  - **Queue** – (Warte-)Schlange, **F**irst-**I**n **F**irst-**O**ut (FIFO).
  - **Stack** – Stapel- oder Kellerspeicher, **F**irst-**I**n **L**ast-**O**ut (FILO).
  - **Set** – Sammlung ohne Duplikate.
- Java bietet für jede Art wiederum verschiedene Implementationen dieser Datenstrukturen an, um den unterschiedlichen Anforderungen gerecht zu werden
  ➔ **J**ava **C**ollection **F**ramework (JCF).
- Hinweis: Eine universelle Datenstruktur, die **alles** perfekt kann, gibt es (leider) **nicht**, wir müssen sie fallspezifisch auswählen!

Eigenschaften von Datenstrukturen

Eigenschaften von Datenstrukturen

- Die verschiedenen Datenstrukturen unterscheiden sich in vielerlei Hinsicht, zum Beispiel:
  - **Grösse**: Dynamisch oder statisch.
  - **Zugriff**: Direktzugriff oder sequenziell.
  - **Sortierung**: Sortiert oder unsortiert bzw. mit/ohne Ordnung.
  - **Suche**: Beschleunigte Suche (z.B. binär oder über Hashwert).
  - **Effizienz**: Der Operationen wie Einfügen, Anhängen, Entfernen, Verschieben etc. einzelner Elemente.
- Der Aufbau der Datenstrukturen und die verwendeten Algorithmen sind zentrale Themen in folgenden Modulen:
  - **AD** (**A**lgo., **D**atenstrukturen und Nebenläufigkeit) für I.
  - **ADS** (**A**lgo. und **D**atenstrukturen **S**ecurity) für AIML und ICS.

Java Collections Framework

Java Collections Framework (JCF) - Übersicht

Das **JCF** besteht aus **drei** wesentlichen Inhalten:

- **Interfaces**: Abstrakte Datentypen, welche verschiedenartige Datenstrukturen repräsentieren und es erlauben, Datenstrukturen unabhängig von ihrer konkreten Implementation zu verwenden.
  - `List<E>` abstrahiert verschiedene Arten von Listen.
- **Implementationen**: Konkrete, wiederverwendbare Implementationen der durch die Schnittstellen definierten Datenstrukturen.
  - `HashMap<K,V>` ist z.B. eine Implementation von `Map<K,V>`.
- **Algorithmen**: Methoden zur Bearbeitung von Datenstrukturen.
  Die Algorithmen sind polymorph implementiert und können somit meist auf verschiedene Datenstrukturen angewandt werden.
  - `sort(List<E>)` sortiert beliebige `List`-Implementationen.

Java Collections Framework - Nutzen

Der Einsatz des Java Collections Framework bietet somit eine ganze Reihe von Vorteilen:

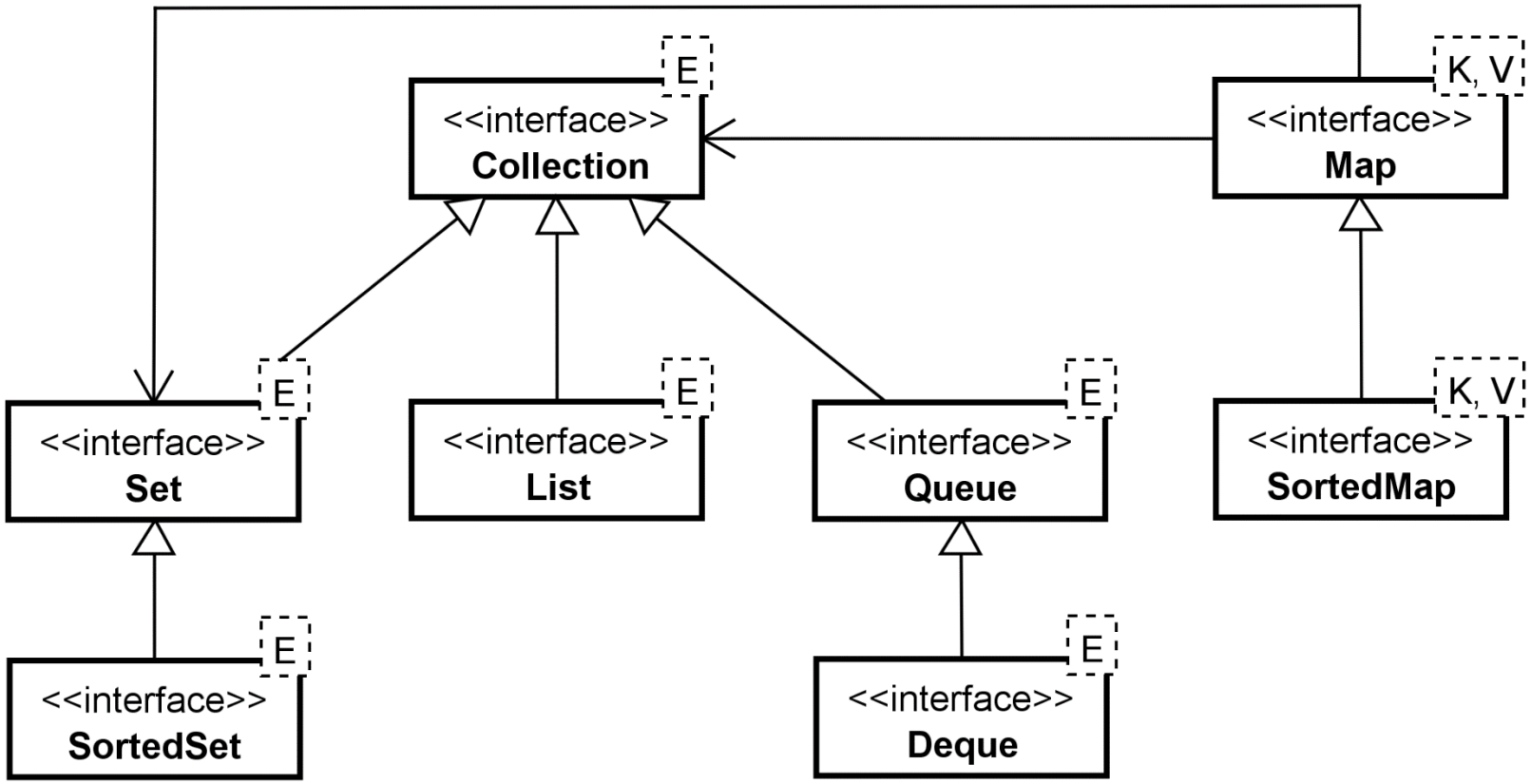
- **Wiederverwendung**: Datenstrukturen und zugehörige Funktionalität müssen nicht selber implementiert werden.
- **Schnell** und (fast) **fehlerfrei**: Die konkreten Implementationen des Frameworks funktionieren sehr zuverlässig und maximal effizient.
- **Austauschbarkeit**: Der Einsatz von Interfaces erleichtert den Austausch und die Wiederverwendung von unterschiedlichen Implementationen von Datenstrukturen ➔Polymorphie.
- **Interoperabilität**: Die breite Verwendung der bereits vorhandenen Collections vereinfacht die Zusammenarbeit durch das schnellere Verständnis massiv.

Java Collections Framework - Generics

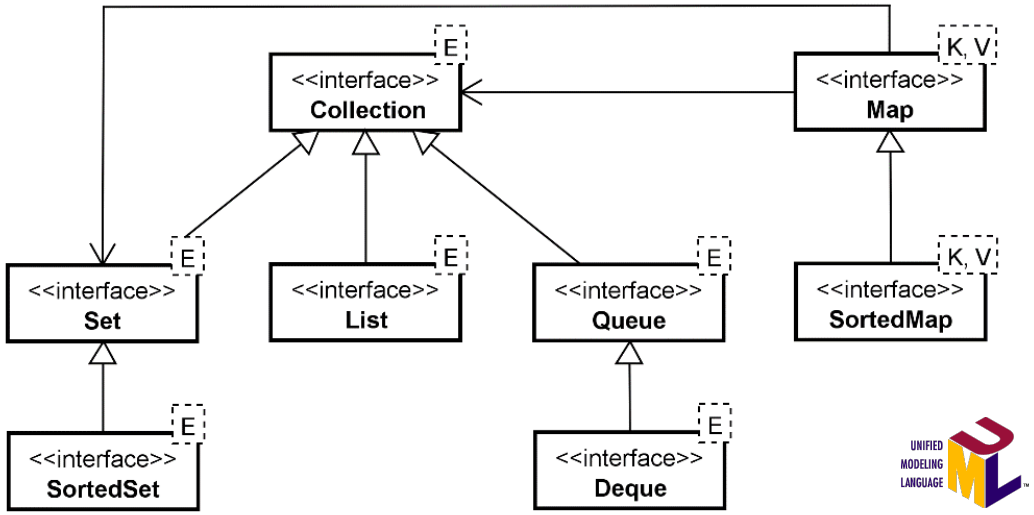
- Bis und mit Java 1.4 waren in Java alle Collections für den Basistyp `Object` implementiert.
  - Es konnten somit beliebige Datenobjekte in die Datenstrukturen eingefügt werden.
  - Bei der Entnahme musste aber immer wieder explizit auf den gewünschten / erwarteten Datentyp gecastet werden.
  - Es waren somit Laufzeitfehler durch falsche Typen möglich.
- Seit der Einführung von ➔Generics in Java 5 (1.5) sind alle Collections **generisch** (re-)implementiert.
  - Massive Verbesserung! Kein explizites Casting mehr notwendig.
  - **Typsicherheit** bereits zur Kompilierzeit gewährleistet!
  - Achtung: Die **raw**-Types (ohne Typparameter) existieren noch immer (wg. Rückwärtskompatibilität), sollten aber **nicht mehr verwendet** werden!

Java Collections Framework – Interfaces - Übersicht

- Auszug aus den vorhandenen **Schnittstellen** (Auszug, nicht vollständig):



Java Collections Framework - Interfaces



- `Collection<E>` – Eine Menge von Objekten vom Typ `E`.
  - `Set<E>` – Keine Duplikate erlaubt, jedes Objekt einzigartig.
    - `SortedSet<E>` – Sortiertes **Set** (natürlich oder speziell).
  - `List<E>` – Geordnete Menge von Objekten mit Direktzugriff.
  - `Queue<E>` – Menge von Objekten für eine (Warte-)Schlange.
    - `Deque<E>` (**d**ouble **e**nded **que**ue) – Spezielle `Queue` mit zusätzlichen Methoden.
- `Map<K,V>` – Paarweise Ablage von Werten (`V`alue) unter einem Schlüssel (`K`ey).
  - `SortedMap<K,V>` – Nach `Key`s sortierte `Map`.

Java Collections Framework - Algorithmen

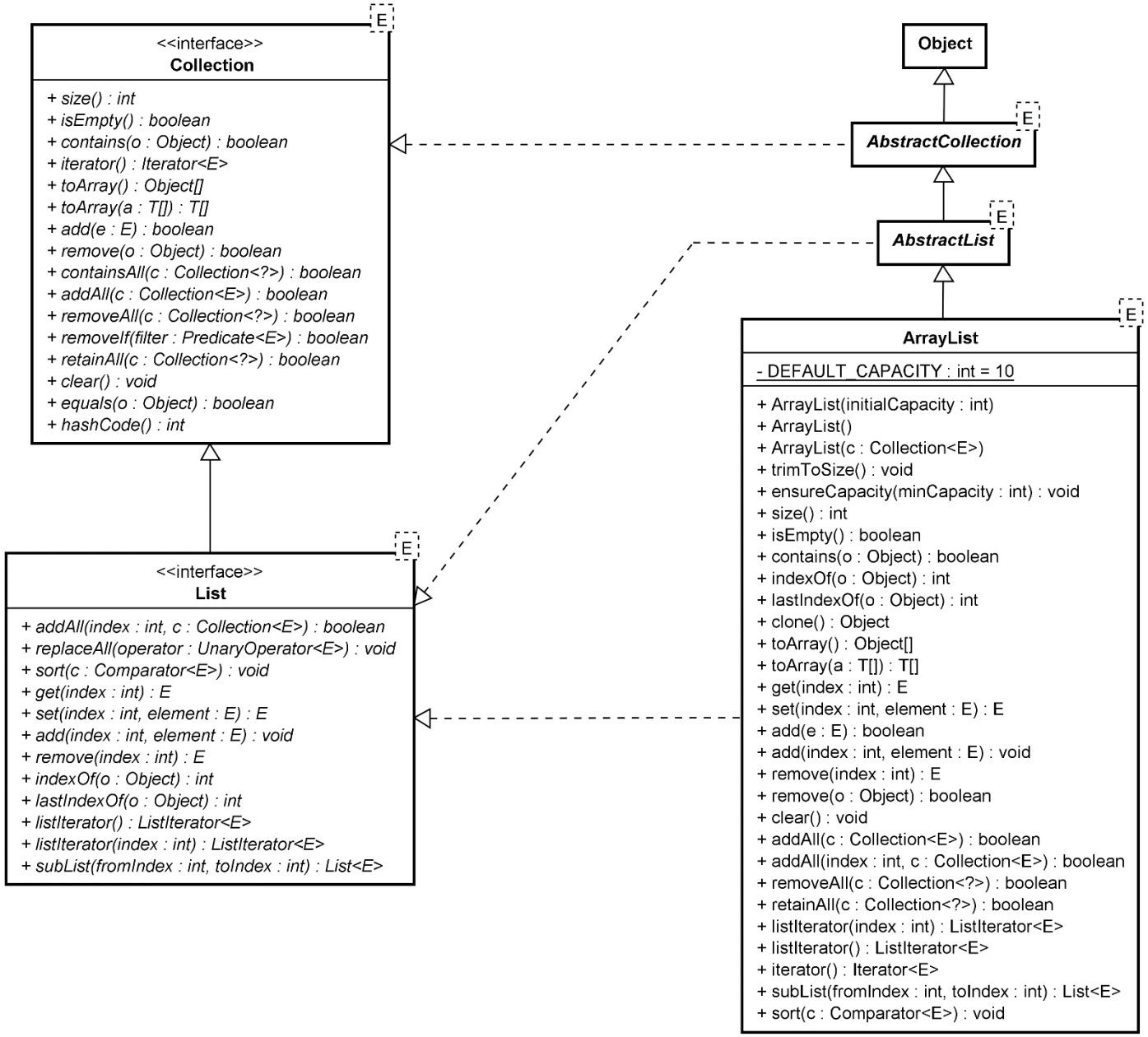
- Die Klasse `Collections` bietet viele statische, **polymorph** implementierte Methoden zur Bearbeitung von Sammlungen an.
- Beispiele (nicht vollständig):
  - `sort(…)` – Schnelles, stabiles Sortieren.
  - `shuffling(…)` – Randomisieren der Reihenfolge.
  - `reverse(…)` – Dreht die Ordnung einer Liste um.
  - `fill(…)` – Füllt eine Liste mit bestimmten Elementen.
  - `copy(…)` – Kopiert eine Liste in eine andere Liste.
  - `swap(…)` – Tauscht die Plätze von zwei Listenelementen.
  - `min(…)/max(…)` – Liefert das kleinste/grösste Element.
  - `frequency(…)` – Zählt ein bestimmtes Element in der Liste.
- **Wichtiger Hinweis**: Nicht alle Funktionen sind für alle möglichen Arten von Datenstrukturen möglich und/oder implementiert!

Java Collections Framework - Funktionsbasis

- Damit die polymorphen Algorithmen und Datenstrukturen des JCF mit **beliebigen** Elementen (Klassen) funktionieren können, benötigen wir ein paar grundlegende Funktionen:
  - Für die Suche von Objekten (oder Ausschluss von Duplikaten) müssen wir Objekte als **«gleich»** (equals) erkennen können.
  - Für die Sortierung müssen wir Objekte vergleichen und als **kleiner** (`<`) oder **grösser** (`>`) beurteilen können.
- Aus diesen Anforderungen resultiert, dass…
  - die Klasse `Object` (Basisklasse) einige dafür notwendige Methoden definiert: `equals()`- und `hashcode()`
  - es Interfaces gibt, die wir pro Klasse/Typ individuell implementieren [müssen]: `Comparable<T>` oder `Comparator<T>`
- Siehe folgender Input ➔ **O10_ObjectEqualsCompare**

Konkretes Beispiel: `ArrayList<E>`

Beispiel: Datenstruktur `ArrayList<E>`



Klasse `ArrayList<E>`: Merkmale

- **Generisch** implementierte Klasse aus dem Package `java.util`
  - Import-Anweisung: `import java.util.ArrayList;`
- **Dynamisch**: Die Grösse einer `ArrayList` passt sich mit zunehmendem Platzbedarf **automatisch** an.
- **Geordnet**: Elemente sind in der `ArrayList` **geordnet** abgelegt.
  - Die Elemente **behalten** ihre Position, sind aber **nicht** automatisch sortiert.
- **Direktzugriff**: Auf einzelne Elemente kann mittels der Methode `get(int index)` **direkt** zugegriffen werden.
  - der Index liegt dabei im Bereich von `0..size()-1`
- Vereinfacht: Die `ArrayList` verhält sich im Prinzip wie ein dynamisches Array.
- Alle Details siehe in der JavaDoc:
  https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/util/ArrayList.html

Beispiel: ArrayList<E> im Einsatz

```java
final List<Temperatur> verlauf = new ArrayList<>();

verlauf.add(new Temperatur(10.2f));
verlauf.add(new Temperatur(15.8f));
verlauf.add(new Temperatur(21.3f));

System.out.println("Anzahl Messwerte: " + verlauf.size());

verlauf.set(1, new Temperatur(18.5f));

final Temperatur t = verlauf.get(1);
System.out.println("Zweiter Messwert: " + t.getCelsius());
```

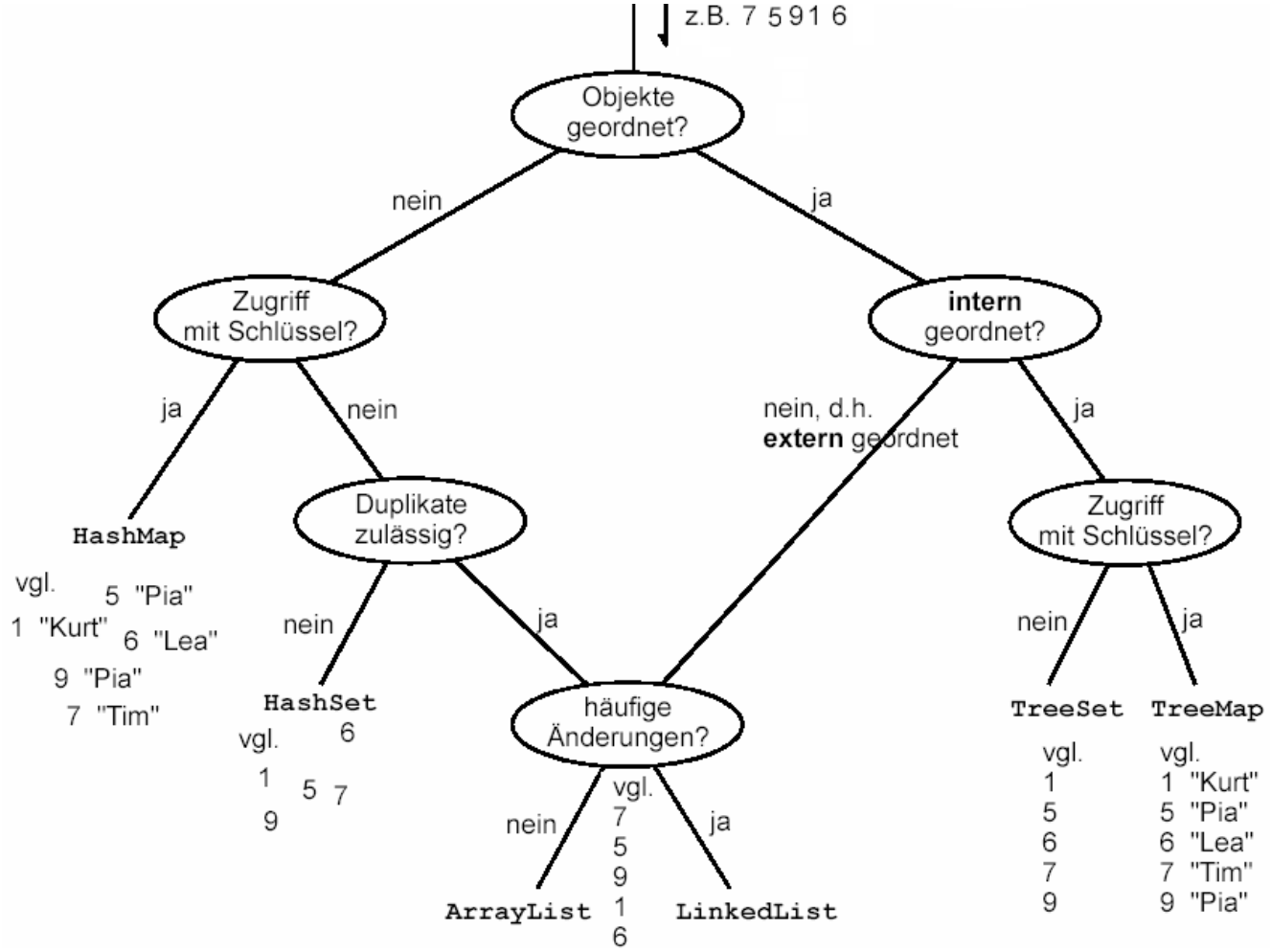
```
Anzahl Messwerte: 3
```

```
Zweiter Messwert: 18.5
```

Auswahl der geeigneten Datenstruktur

Auswahl der geeigneten Datenstruktur

- Der folgende Entscheidungsbaum dient Ihnen als erste **grobe** Richtschnur zur Auswahl einer geeigneten Datenstruktur:



Zusammenfassung

- Datenstrukturen dienen zur Verarbeitung von Mengen von Daten bzw. Objekten.
- Es gibt viele verschiedene Datenstrukturen mit unterschiedlichen Fähigkeiten und Eigenschaften.
- Das Java Collection Framework enthält Schnittstellen, Algorithmen und konkrete Implementationen für und von Datenstrukturen.
- `ArrayList<E>` ist ein Beispiel einer sehr häufig eingesetzten, relativ universellen Datenstruktur in Java.
- Die Auswahl der geeigneten Datenstruktur erfolgt aufgrund der konkreten Anforderungen.
  - Kriterien wie Datenmenge, Ordnung, Semantik, Zugriffsart, Mutationshäufigkeit etc.

Fragen?